

CS-499: 4-2 Enhancement Two Algorithms and Data Structure

Syed Zeeshan Hyder

Southern New Hampshire University

CS-499: Computer Science Capstone

Professor Joseph Martinez

July 16th, 2026

Briefly describe the artifact. What is it? When was it created?

The artifact I selected for this assignment and the next two is the TravlR Getaways Full-Stack Web Application, originally created during CS 465: Full Stack Development 1, a class I took in my previous term, May - June. The project, put simply, is a full-stack travel booking application built using the MEAN stack, consisting of an Angular administrator application, a Node.js and Express backend, and a MongoDB database accessed via Mongoose. The application also uses JWT authentication, which enables administrator access management, and follows the MVC architecture, which separates routes and controllers.

At the time of submitting the original application, it did meet the requirements presented in CS 465, but looking at it through the fresh lens being this capstone allowed me to evaluate it from a professional software engineering perspective. Instead of building a bunch of new functionality, this enhancement focuses on improving the application's algorithms by implementing searching, sorting, and pagination. These improvements make it easier to organize and retrieve information while allowing the application to scale more effectively as the number of trips grows.

Justify the inclusion of the artifact in your ePortfolio. Why did you select this item? What specific components of the artifact showcase your skills and abilities in algorithms and data structure? How was the artifact improved?

The reason I chose this artifact for a second time is that it gives me the chance to work on one major application and have it stand out, as opposed to minor changes on artifacts that wouldn't get the attention of anyone. This artifact shows how algorithmic thinking is able to improve the usability and efficiency of a full-stack application, and unlike many other assignments within my academic journey, which focus on standalone algorithmic problems, this assignment alone demonstrates how searching, sorting, and pagination are all done in a real-world application that communicates between the frontend, backend, and database.

The primary goal for this enhancement in milestone two was redesigning the trip listing functionality so it would be able to support searching, sorting, and pagination. The search was added using MongoDB regular expression queries, and this gave users the ability to search for available trips by their name, code, or resort, without needing to be an exact match. For example, if 'reef' was typed, this returned all three trips with 'reef' in the title. Sorting allowed users to organize results by different fields in either ascending or descending order, and finally pagination was used to limit the number of records which were returned at one time, and allowed users to navigate between multiple pages of results.

To add these enhancements, I had to modify both the backend and the frontend, where the backend now constructs MongoDB queries based on user-selected search and sorting parameters before returning only the requested page of results. The frontend is able to communicate these parameters through HTTP requests and would then update the displayed results whenever the search text, sorting options, or page changes.

Did you meet the course outcomes you planned to meet with this enhancement in Module One? Do you have any updates to your outcome-coverage plans?

I would say that yes, this enhancement successfully met the outcomes that I planned to address for the algorithms and data structures category in Module One. The primary outcome demonstrated through this enhancement is the ability to design and evaluate computing solutions that solve problems using

algorithmic principles and computer science practices while managing the trade-offs involved in design decisions.

Instead of retrieving every single trip in the database and relying solely on the frontend to organize the results to the user, the artifact now performs searching, sorting, and pagination in the backend prior to returning only required records back to the user. Thinking of any big-name website or application where the user has to choose between options, whether it be shows, trips, or games, they are usually given the ability to filter out to what they need, and this approach reduces the unnecessary transfer of data, improves scalability, and makes my artifact better align with how modern web applications handle their large datasets.

I would also add how this enhancement bettered my ability to communicate technical concepts through documentation like this one, as explaining why particular algorithms and how they were implemented is an important part of professional software development.

At this point in time, my enhancement plan remains unchanged, and the final enhancement will focus on improving the application's database design through schema improvements and performance optimization, while adding to the software engineering and algorithmic enhancements completed throughout the capstone.

Reflect on the process of enhancing and modifying the artifact. What did you learn as you were creating it and improving it? What challenges did you face?

Through this project, and specifically this milestone, I learned how algorithms can be brought into a full-stack application to improve the overall user experience while reducing unnecessary processing and network traffic. At first, I didn't think that algorithms were needed for web apps, but seeing the jump in efficiency it made on Travlr, when the artifact was once considered 'done,' makes me think about what else I can incorporate algorithms into within the application. I learned that algorithm design is much more than producing the correct result, and that it also requires decision-making on where those algorithms should execute. At first, it was easier to simply retrieve every trip regardless of what the user wanted, but then I implemented searching and sorting operations on the backend so only the required records are sent to the client, which makes the application more scalable but also a better professional web application.

I think the most challenging part of this enhancement was making sure that the searching, sorting, and pagination methods all worked together correctly, as I got each feature to work independently, but being able to combine them took careful coordination between the frontend interface, HTTP requests, and MongoDB queries. During development, I had issues where searching returned no results, or pagination was acting strangely, which required careful debugging and testing to find each problem. I think the most annoying issue I faced was how typing gibberish such as 'skfhuh' or any other random jumble of letters would still return a trip with the same starting letter. With the jumble of letters from before, starting with an S, the trip record for 'Saudi Arabia' was still returned, and it took a long time to fix.